

Serverless Notebook Runbook

A developer's pre-flight checklist for cost-aware notebook work on Databricks

Circle K · Data & Analytics

v1.0 · May 2026

WHY THIS EXISTS

Serverless removes *infrastructure* friction — no cluster sizing, no boot time, no idle minutes. The trade-off is that the developer is still the person deciding how much data to scan and how long to keep a session alive. Serverless adoption tends to grow quickly once a team gets a taste of it, and the cost shape shifts with it: from a few long-running clusters to many small queries running constantly. This runbook is the short list of habits that turn the flexibility into efficient outcomes.

WHAT GOOD LOOKS LIKE

- You know the rough scan size *before* you hit Run.
- Notebooks detach when you walk away.
- `SELECT *` never reaches a billion-row fact table.
- Big queries get `EXPLAIN` 'd first.
- Exploration uses `LIMIT` or `TABLESAMPLE` .

WHAT COSTS US MONEY

- Idle sessions running overnight or over weekends.
- `SELECT *` + no `WHERE` on partitioned tables.
- Running cells in a loop to "see what happens".
- Re-running the same exploration instead of caching a sample.
- No `WHERE` clause on the cluster-key column.

1 · Quick reference card

The five patterns that account for most of the cost surprises, and the buttons to hit when something is already running away. For the ritualized "did I do my checks today?" view, see §2.

The five do/don't pairs

Don't	Do
<code>SELECT * FROM huge_fact</code>	List the columns you actually need. Column pruning happens automatically — but only if you name them.
<code>SELECT COUNT(*) FROM huge_fact WHERE date_str = '2026-05-18'</code> (<i>no partition pruning</i>)	Filter on the partition or clustering column with the right type: <code>WHERE event_date = DATE '2026-05-18'</code> .
Run an unbounded exploratory query repeatedly	Sample first: <code>SELECT * FROM t TABLESAMPLE (1 PERCENT) LIMIT 10000</code> — then iterate on the sample.
Leave the notebook attached overnight while a cell is "thinking"	Detach. Serverless default kills queries after 2.5 h, but you can stop them yourself in 2 seconds.
Skip the query profile — "it ran fine"	Open <i>See performance</i> . If the scan node says 100 GB, your query is expensive even if it returned in 30 s.

Emergency stop

Situation	What to do
Cell has been running for > 5 min and you didn't expect it to	Click the stop button on the cell. Cost stops immediately.
You stepped away and want to make sure nothing's running	Top-right compute pill → Detach & Re-attach closes the session.
You suspect a runaway query but the cell is unresponsive	Refresh the page. The serverless backend will keep running for up to 2.5 h. Then go to Query History and cancel from there.
Cap a notebook to a shorter timeout	Add to a Python cell: <code>spark.conf.set("spark.databricks.execution.timeout", "1800")</code> (30 min, in seconds).

DEFAULT OVERSPEND GUARD

Databricks kills serverless notebook queries at **2.5 hours** by default. That's a safety net, not a budget — a 2-hour `SELECT *` on a 10 TB table will still complete (and bill) before the timeout fires.

2 · Daily checklist (print & pin)

§1 is the cheat-sheet you glance at when something feels off. This is the discipline you run on autopilot. Stick it on the second monitor.

Before opening a notebook

- ☐ I know **which table** I'm querying and roughly **how big** it is (run `DESCRIBE DETAIL table` or check Catalog Explorer).
- ☐ I know the table's **partition column** or **cluster keys** (Catalog Explorer → Details).
- ☐ If the task is exploratory, I plan to **sample first**, not query the full table.

Before running a SQL/Spark cell

- ☐ I named specific columns instead of `*`.
- ☐ I have a `WHERE` clause on the partition / cluster key.
- ☐ I have a `LIMIT` on every exploratory `SELECT`.
- ☐ If the query joins, aggregates, or scans > 10 GB, I ran `EXPLAIN` first and confirmed the filter pushed down.
- ☐ For wide aggregations: I considered whether a **Metric View** already answers this.

After a cell finishes

- ☐ I clicked **See performance** and glanced at the scan node — bytes read is a fraction of the table size.
- ☐ If it scanned more than I expected, I revised the query before running the next variation.

Before leaving for the day

- ☐ No cells are running.
- ☐ I detached the notebook from compute (compute pill → Detach).
- ☐ I closed notebook tabs I don't need open tomorrow.

Weekly (5 min)

- ☐ I checked **Query History** for my long-running or high-scan queries.
- ☐ I have no notebooks from departed projects still attached to me.

3 · Top anti-patterns on serverless notebooks

3.1 `SELECT *` on a fact table

Column pruning is automatic on Delta — but only for columns you don't reference. `SELECT *` tells the engine "I need everything", so it reads every column file from every row group. On a 200-column fact table, this is often 10–50× more I/O than necessary.

BAD

```
SELECT *  
FROM gold.sales_fact  
WHERE event_date = DATE '2026-05-18'
```

GOOD

```
SELECT store_id, product_id, units,  
       revenue  
FROM gold.sales_fact  
WHERE event_date = DATE '2026-05-18'
```

3.2 Filtering on a column the table isn't partitioned by

If `sales_fact` is partitioned (or clustered) by `event_date`, filtering by `order_id` alone forces a full table scan. Always include the partition / cluster column in your `WHERE` clause if you know its value.

```
-- bad: full scan, even though we only want one order  
SELECT * FROM gold.sales_fact WHERE order_id = 9182734;  
  
-- good: prune to one day, then filter  
SELECT * FROM gold.sales_fact  
WHERE event_date = DATE '2026-05-18' AND order_id = 9182734;
```

3.3 String literals against typed columns

Comparing a `DATE` column to a `STRING` like `'2026-05-18'` sometimes prevents partition pruning depending on the engine's cast handling. Use the matching type explicitly: `DATE '2026-05-18'`, `TIMESTAMP '...'`.

3.4 Idle sessions

Serverless bills per second of *active* compute, but a notebook still consumes resources while idle if a query is running or pending. The trap is leaving a long-running cell going while you head into a meeting. **Stop the cell, not just the tab** — closing the tab doesn't cancel a running query.

3.5 Re-running the same exploration

If you've queried the same slice three times in 10 minutes, materialize it:

```
df = spark.sql("""  
    SELECT ... FROM gold.sales_fact  
    WHERE event_date BETWEEN DATE '2026-05-01' AND DATE '2026-05-15'  
    """).cache()  
df.count() # forces the cache  
# now iterate on df locally — no more re-scanning the fact table
```

3.6 Joining without filters

Two 1 TB tables joined unfiltered is a 1 TB+ shuffle. Always push down filters into both sides of the join. Run `EXPLAIN` and look for filter nodes *below* the join — not above it.

3.7 Putting business math inside Genie / notebook prompts

If a calculation needs to be consistent and trusted, define it as a **Metric View**. LLMs and ad-hoc notebooks both get the math wrong silently. Metric Views centralize the definition and Genie / dashboards both inherit it.

4 • How serverless notebook billing actually works

You're billed per second of active serverless compute used by your queries. The unit is the **serverless DBU**, charged at the rate for your region and SKU (e.g. `PREMIUM_ALL_PURPOSE_SERVERLESS_COMPUTE_*`). There is *no* idle charge when no queries are running — but a running or pending query consumes resources.

What drives the bill, in order

1. **Bytes scanned.** The dominant cost driver. More data = more parallelism = more DBUs.
2. **Total active query time.** Long queries cost more, even on identical scans.
3. **Concurrency.** Each cell you run in parallel allocates additional compute.
4. **Spill.** If a join shuffle doesn't fit in memory, the engine spills to disk and the query gets slower — and more expensive.

Serverless vs. classic — when each wins

Pattern	Serverless	Classic all-purpose
Quick ad-hoc exploration, infrequent	Wins — instant start, no idle cost	Loses — boot time + idle minutes
Multi-hour interactive session, continuous work	Roughly equal — but no risk of forgetting to shut down	OK if you actually use it; bad if it sits
Big repeated batch over the same data	Loses — pays per second every run	Wins with disk cache warmed; or use Jobs compute
Scheduled / repeatable workload	Use Jobs serverless instead — ~50% cheaper rate than interactive serverless	—

5 • Diagnostics — read the plan, then read the profile

Two tools tell you whether a query is doing what you think. **EXPLAIN** shows the plan *before* you run; the **query profile** shows what actually happened *after*. On serverless the Spark UI is unavailable, so these are your only debugging surfaces.

5.1 EXPLAIN — 60-second pre-flight

EXPLAIN shows the query plan without running the query. The goal is to read three things:

1. **PartitionFilters / DataFilters** at the scan node — confirms your `WHERE` reached the storage layer.
2. **The size estimate** — Databricks shows estimated rows/bytes for scans and joins. A big surprise here means your filter isn't doing what you think.
3. **Photon** nodes — for SQL, you want to see `Photon` in the plan; if not, you're missing the JIT speedup.

```
EXPLAIN FORMATTED
SELECT store_id, SUM(revenue) AS rev
FROM gold.sales_fact
WHERE event_date BETWEEN DATE '2026-05-01' AND DATE '2026-05-15'
GROUP BY store_id;
```

Look for, on the scan node:

```
PartitionFilters: [isNotNull(event_date#42),
                  event_date#42 >= 2026-05-01,
                  event_date#42 <= 2026-05-15]
```

If `PartitionFilters` is empty *and* the table is partitioned by date, your filter isn't pushing down. Common causes: wrong type cast, function on the column (`WHERE YEAR(event_date) = 2026` defeats pruning), filter applied after a join.

5.2 Query profile — post-flight

After every cell on serverless compute, Databricks shows a **See performance** link. Click it. Then click any statement to open the **query profile** — a visualization of every operator the engine ran. Spend 30 seconds on it after every meaningful query.

Metric	Where	Healthy	Red flag
Bytes read	Scan node	Tiny fraction of table size	Whole table — filter not pushed
Files read / pruned	Scan node	Pruned » Read	Pruned = 0 — partition / clustering not used
Rows shuffled	Exchange node	≤ rows in output	Billions for a small result — missing pre-filter
Spill (memory)	Aggregate / SortMergeJoin	0 B	GBs — query is too big for this size, reduce scope
Photon	Top of profile	Photon ✓	Operator not in Photon — usually a UDF; rewrite in SQL/native

RULE OF THUMB

If the actual *bytes read* from the profile is more than ~10% of the table size, your filter isn't doing its job — go back to `EXPLAIN` and find out why.

6 · Reducing scan size — your single biggest lever

6.1 Use the partition / cluster column in your WHERE

Check the table's definition in Catalog Explorer. Liquid-clustered tables show their cluster keys; partitioned tables show partition columns. Always filter on at least one of these for any non-trivial query.

6.2 Filter *before* joins, not after

```
-- bad: join all of t1 to all of t2, then filter
SELECT ... FROM t1 JOIN t2 ON t1.id = t2.id WHERE t1.event_date = DATE '2026-05-18';

-- good: filter both sides before the join
SELECT ... FROM
  (SELECT * FROM t1 WHERE event_date = DATE '2026-05-18') a
  JOIN
  (SELECT * FROM t2 WHERE event_date = DATE '2026-05-18') b
  ON a.id = b.id;
```

6.3 Don't wrap filtered columns in functions

`WHERE DATE_TRUNC('day', event_ts) = DATE '2026-05-18'` defeats pruning because the engine can't push `DATE_TRUNC` down. Use a range instead: `WHERE event_ts >= TIMESTAMP '2026-05-18 00:00:00' AND event_ts < TIMESTAMP '2026-05-19 00:00:00'`.

6.4 Sample for exploration

```
-- 1% sample, capped at 10k rows
SELECT * FROM gold.sales_fact TABLESAMPLE (1 PERCENT) LIMIT 10000;

-- deterministic by hash
SELECT * FROM gold.sales_fact WHERE HASH(order_id) % 100 = 0;
```

6.5 Cache once, iterate many

```
df = spark.read.table("gold.sales_fact") \
  .filter("event_date = DATE '2026-05-18'") \
  .select("store_id","product_id","units","revenue") \
  .cache()
df.count() # materialize
```

6.6 Predicate pushdown across federation

If you're querying a federated source (Snowflake / Iceberg foreign catalog), check the query profile for a *RemoteScan* node — Databricks pushes filters and column lists to the source. If the source's plan shows a full scan, the predicate isn't pushing — file a ticket with the predicate signature.

7 • Session hygiene

7.1 Detach explicitly

Closing a tab does not stop a running query. Walk through this checklist when you leave for the day:

- 1 Scan the notebook list view for **green running indicators**.
- 2 For any green: open and stop the cell.
- 3 Click the compute pill → Detach & Re-attach to clear state.
- 4 Close the tab.

7.2 Cap the per-query execution timeout

```
# Add this as the first cell in any heavyweight notebook.  
# Cap any single Spark query at 30 minutes (1800 seconds).  
spark.conf.set("spark.databricks.execution.timeout", "1800")
```

Default is **9000 seconds (2.5 h)**, applied per Spark query (not per session or per notebook). A long-running Python loop that issues many short Spark queries is *not* bounded by this — it only fires when a single query exceeds the limit. For exploration notebooks, lowering this to 15–30 minutes is a good guard against an accidental full-table scan.

SCOPE IS PER-NOTEBOOK, NOT WORKSPACE-WIDE

Databricks does **not** currently support setting Spark configurations workspace-wide for serverless interactive notebooks — only per-notebook via `spark.conf.set(...)`. Init scripts aren't supported on serverless, and workspace base environments cover Python packages only. To roll this cap out to every developer, plant the `spark.conf.set(...)` line in a shared `%run ./common/init` notebook for interactive work, and set `spark.databricks.execution.timeout` in the job environment block for anything scheduled.

Reference: Set Spark configuration properties on Databricks — see the `spark.databricks.execution.timeout` entry.

7.3 Use unique temp view names in loops

Serverless uses Spark Connect, which evaluates temp views lazily. Reusing the same view name in a loop (e.g. `tmp`) causes weird late errors. Suffix with a counter or UUID.

7.4 Pin Python package versions

```
# requirements.txt  
pandas==2.2.3  
numpy==2.2.2  
scikit-learn==1.5.2
```

Unpinned packages re-resolve on every cold start, adding seconds of latency that — at scale across the team — accumulate into noticeable cost.

8 · For platform leads — governance levers

The developer practices above are necessary but not sufficient. The platform team should layer these guardrails on top.

8.1 Mandatory tags via budget policies

Every serverless workload should attach to a **budget policy** with at minimum:

- `business_unit` (e.g. supply-chain, loyalty, store-ops)
- `project_name`
- `environment` (dev / qa / prod)

Tags are propagated to `system.billing.usage` for attribution. Without them you can't tell who's spending what.

8.2 Email alerts on serverless interactive usage

Set budget-policy alerts that trigger on DBU consumption — calibrate the thresholds against last quarter's per-user and per-BU baseline. A three-tier model works well:

Alert	Per user (weekly DBU)	Per BU (weekly DBU)
Heads-up	~2× the team median	~1.2× the BU's prior-quarter weekly average
Investigate	~5× the team median	~1.5× the BU's prior-quarter weekly average
Stop & review	~10× the team median	~2× the BU's prior-quarter weekly average

Anchor on relative usage rather than absolute numbers so the thresholds don't go stale as the team grows.

8.3 Soft cap, not hard cap (initially)

Hard-stop budget policies will ship in June. Until then — and even after — start with email alerts and weekly reviews, not hard stops. A surprise hard cap mid-query damages trust; a Monday email asking "what is this query?" doesn't.

8.4 Recurring > ad-hoc

Any notebook that runs more than 3× per week on a schedule is a Job, not a notebook. Move it to **Jobs serverless (Standard mode)** — Databricks documents up to **70% cost reduction** vs. performance-optimized mode, since job startup latency doesn't matter.

8.5 Replace exploratory dashboards with Metric Views + Genie

Repeated ad-hoc "let me check the numbers" sessions are a signal that a Metric View is missing. Once defined, the same metric is reusable in dashboards, Genie spaces, and the API — and only computed once.

8.6 Weekly cost-optimization review

- Top 10 users by serverless interactive DBU last week.
- Top 10 longest queries (> 30 min).

- Top 10 highest-scan queries (> 1 TB read).
- Compare WoW. Reach out to anyone in the top 3 with a friendly note.

9 • Appendix — useful snippets

Find your heaviest queries from the past 7 days (system tables)

```
SELECT
  executed_by_user_email,
  statement_text,
  ROUND(total_duration_ms / 1000.0 / 60.0, 1) AS minutes,
  ROUND(read_bytes / 1024.0 / 1024.0 / 1024.0, 1) AS gb_read,
  read_files,
  read_partitions
FROM system.query.history
WHERE start_time >= CURRENT_DATE() - INTERVAL 7 DAYS
  AND executed_by_user_email = current_user()
  AND compute.type IN ('SERVERLESS_INTERACTIVE', 'SERVERLESS_NOTEBOOK')
ORDER BY total_duration_ms DESC
LIMIT 20;
```

Top 10 serverless interactive users this month (platform team)

```
SELECT
  identity_metadata.run_as AS user_id,
  ROUND(SUM(usage_quantity), 0) AS dbus,
  COUNT(DISTINCT usage_metadata.notebook_id) AS notebooks_used,
  COUNT(DISTINCT DATE(usage_start_time)) AS active_days
FROM system.billing.usage
WHERE usage_start_time >= DATE_TRUNC('month', CURRENT_DATE())
  AND billing_origin_product = 'INTERACTIVE'
  AND usage_type = 'SERVERLESS_COMPUTE'
GROUP BY identity_metadata.run_as
ORDER BY dbus DESC
LIMIT 10;
```

Notebooks idle > 30 min with a session still open

```
SELECT
  session_id,
  notebook_path,
  user_name,
  MAX(end_time) AS last_activity,
  TIMESTAMPDIFF(MINUTE, MAX(end_time), CURRENT_TIMESTAMP()) AS minutes_idle
FROM system.query.history
WHERE start_time >= CURRENT_DATE() - INTERVAL 1 DAY
  AND compute.type LIKE 'SERVERLESS%'
GROUP BY session_id, notebook_path, user_name
HAVING minutes_idle > 30
ORDER BY minutes_idle DESC;
```

Drill down with Genie

The SQL above gives you the top-N rollup. For the follow-up questions — "split that user's spend by table", "which days saw the largest scan-byte spike", "who runs `SELECT *` the most" — point a Genie space at `system.query.history` and `system.billing.usage` and ask in plain English. It's faster than iterating on SQL by hand, and the answers stay consistent with any Metric Views you've already defined (§8.5).

References (internal)

- Best practices for serverless compute · docs.databricks.com/aws/en/compute/serverless/best-practices
- Serverless compute for notebooks · docs.databricks.com/aws/en/compute/serverless/notebooks
- Best practices for cost optimization · docs.databricks.com/aws/en/lakehouse-architecture/cost-optimization/best-practices
- Interactive Cluster Downsizing Guide — Circle K (Dong Qiao Yang, Apr 2026)
- Serverless Compute Field Guide · [go/serverless/jobs-field-guide](https://go.databricks.com/serverless/jobs-field-guide)
- SQL Performance Tuning Best Practices Guide · Confluence page 5573509125